

# Graph Neural Tangent Kernel: Convergence on Large Graphs

Shivam Kumar (22b0917)  
Lovesh Mahar (22b0975)  
Nihar Hingrajia (22b1044)  
Suryansh Srijan (22b1064)

Github Link: <https://github.com/SuryanshSrijan/wntk/tree/lwg/fix>

## Initial Problem Statement

This project focuses on implementing and debugging the Graph Neural Tangent Kernel (GNTK) model for node classification tasks using the Planetoid benchmark datasets: **Citeseer, Cora, and Pubmed**. In these datasets, each node is assigned a class label based on its feature vector as well as the underlying graph structure (unlike assigning a class label to the entire graph). Our goal is to reproduce and validate the findings of Krishnagopal et al. (2023), who demonstrate that GNTK converges to a Graph Neural Network (GNN) in the infinite-width limit.

## Literature Review

To build a strong theoretical foundation, we studied the work of Krishnagopal et al. (2023), which forms the basis of our project. Additionally, we reviewed Graph Neural Tangent Kernel: Fusing Graph Neural Networks with Graph Kernels by Du et al. (2019) to understand the original GNTK framework. Foundational materials on **Neural Tangent Kernels** *NTK* and **graphons** were also studied to acquire the necessary background knowledge.

After establishing the theoretical groundwork, we moved to the implementation phase. However, the provided codebase from Krishnagopal et al. (2023) proved difficult to work with—it was non-functional, poorly documented, and filled with debug statements. The overall code structure and coding practices made it difficult to understand the given implementation. To address these issues, we completely rewrote the codebase, focusing on verifying the central claim: that a GNTK trained on a small subgraph can still achieve high accuracy when making predictions on the full test graph.

## Revised Problem Statement

Our revised objective is to replace the unstable and non-functioning implementations of the GNN and GNTK models from Krishnagopal et al. (2023) with the more robust and well-documented versions from Du et al. (2019). Using this improved setup, we aim to train both models on subgraphs of varying sizes and evaluate their performance on two fronts: (1) node classification within the test subgraph and (2) generalization to the entire test graph. This investigation allows us to assess the transferability and generalization capabilities of both GNTK and GNN models across different regions of the graph.

## Methodology

Our implementation consists of two parallel components:

- **GNN Model:** A multi-layer graph neural network with:
  - **Graph convolution layers with ReLU activation:** Each GNN layer performs kernel-based message aggregation followed by a linear transformation and optional nonlinearity. The update rule is given by:

$$H^{(l+1)} = \sigma \left( W^{(l)} (K H^{(l)}) \right)$$

where  $K$  is a precomputed node similarity (kernel) matrix,  $W^{(l)}$  is a trainable weight matrix, and  $\sigma$  denotes a non-linear activation function such as ReLU.

- **Jumping knowledge connections (when enabled):** Jumping knowledge combines intermediate layer outputs:

$$h_v^{\text{final}} = \text{JK}(h_v^{(1)}, h_v^{(2)}, \dots, h_v^{(L)})$$

where  $\text{JK}(\cdot)$  denotes a layer aggregation strategy, such as concatenation or max-pooling over layers.

- **Final MLP classification head:** After graph-level readout  $h_G$ , we apply a standard MLP:

$$\hat{y} = \text{MLP}(h_G)$$

where  $h_G = \text{READOUT}(\{h_v^{\text{final}}\}_{v \in G})$  using sum or mean pooling.

- **GNTK Model:** The corresponding neural tangent kernel with:

- **Recursive kernel computation through network layers:** The kernel between two graphs  $G$  and  $G'$  is computed recursively. For node features  $h_v^{(l)}$ , the base kernel is initialized as:

$$\Sigma_{v,v'}^{(0)} = h_v^{(0)} \cdot h_{v'}^{(0)}, \quad \dot{\Sigma}_{v,v'}^{(0)} = \Sigma_{v,v'}^{(0)}$$

Then recursively:

$$\Sigma_{v,v'}^{(l+1)} = \mathbb{E}_{(z,z') \sim \mathcal{N}(0, \Sigma_{v,v'}^{(l)})} [\sigma(z)\sigma(z')], \quad (\text{nonlinear propagation})$$

$$\dot{\Sigma}_{v,v'}^{(l+1)} = \dot{\Sigma}_{v,v'}^{(l)} \cdot \mathbb{E}_{(z,z')} [\dot{\sigma}(z)\dot{\sigma}(z')]$$

- **Exact gradient flow dynamics:** The GNTK combines the forward and gradient paths as:

$$\Theta_{v,v'}^{(l+1)} = \Theta_{v,v'}^{(l)} + \dot{\Sigma}_{v,v'}^{(l+1)}$$

For node classification, we use  $\Theta_{v,v'}^{(L)}$  directly without summing over nodes.

- **Logistic regression head for classification:** After computing the kernel matrix  $K \in \mathbb{R}^{n \times n}$ , a logistic regression is trained using:

$$\hat{y} = \sigma(K\alpha)$$

where  $\alpha$  is learned by minimizing the logistic loss:

$$\mathcal{L} = - \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

## Training Protocol

For each experiment we:

1. Sample subgraphs of varying sizes (200-2000 nodes)
2. Train both models on identical subgraphs
3. Evaluate on:
  - Held-out nodes from the subgraph (test accuracy)
  - Full graph nodes (transfer accuracy)
4. Repeat across 5 realizations statistical significance

## Experiments

We tested our code on the following architectures:

```
GNN_hidden_dims: [10, 50, 100, 1000]
GNN_num_layers: [2, 3]
GNN_num_mlp_layers: [1]
```

For each combination (8) of the following, the start layer has F\_in as input dim, and F\_out as hidden\_dim, the middle layers have [hidden\_dim x hidden\_dim] and last layer has [hidden\_dim x num\_classes], also for each layer there are num\_mlp\_layers of perceptron between them.

The GNTK is supposed to work as an infinite-width GNN that is trained with a low learning rate for a large number of epochs. The GNN with higher dims could not train well; however, the GNTK gave a good enough test accuracy and transfer accuracy. The test and transfer accuracies of the GNN architectures and GNTK are given dataset-wise in the appendix. Here are some highlighted results for each dataset.

## Cora

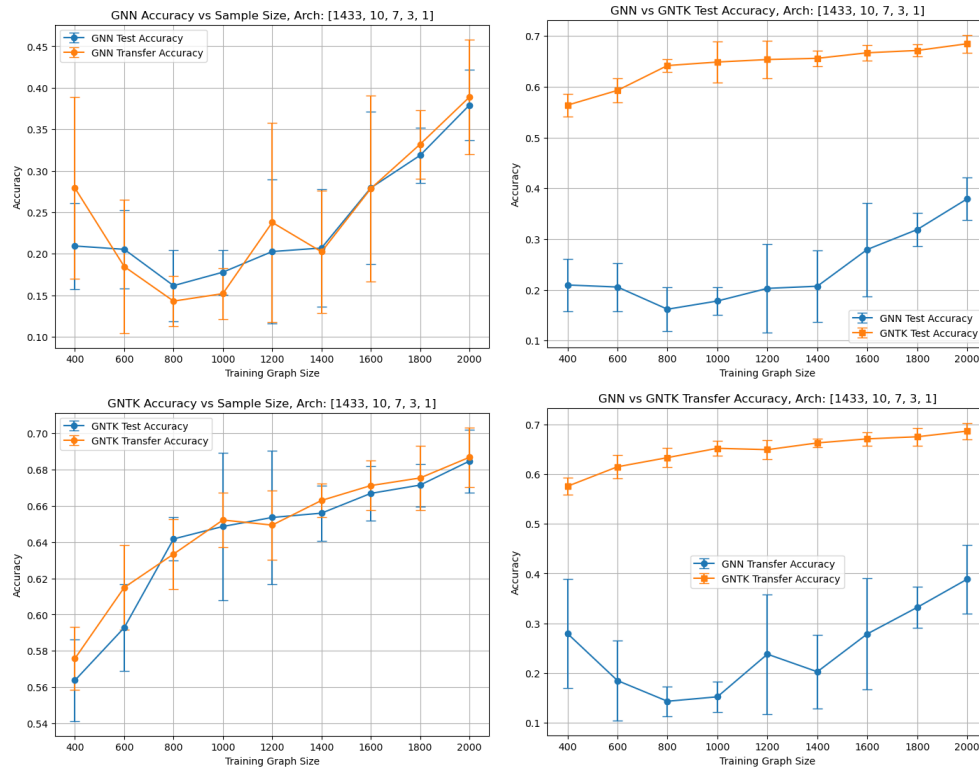


Figure 1: Results on Cora dataset with num\_layers=3, and num\_hidden\_layers=10

## Citeseer

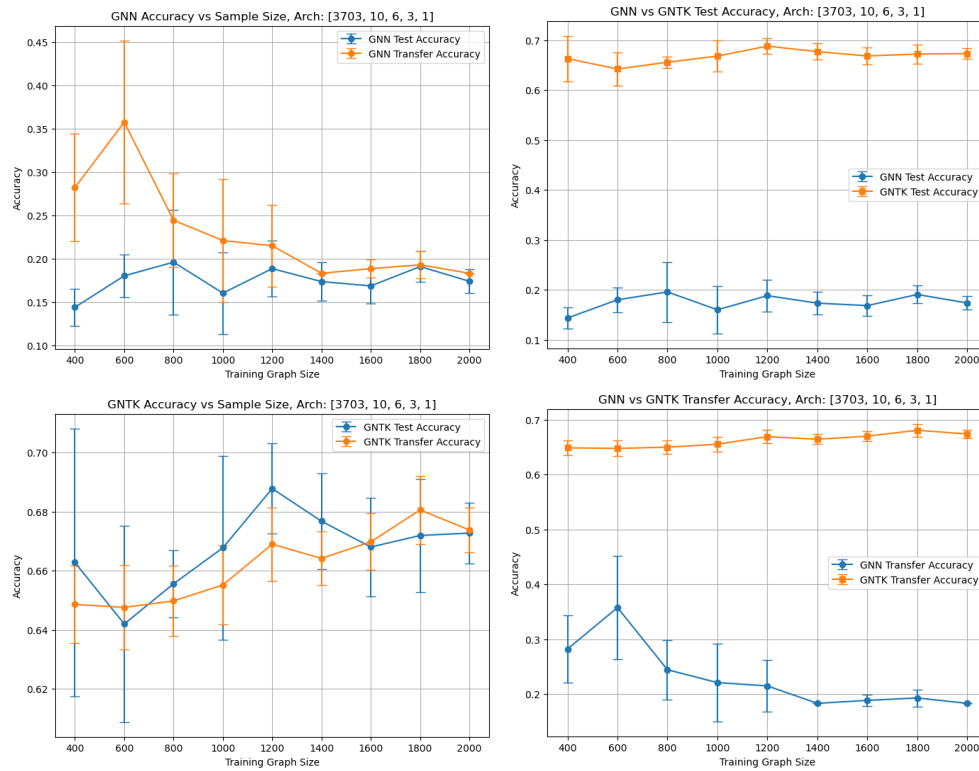


Figure 2: Results on Citeseer dataset with num\_layers=3, and num\_hidden\_layers=10

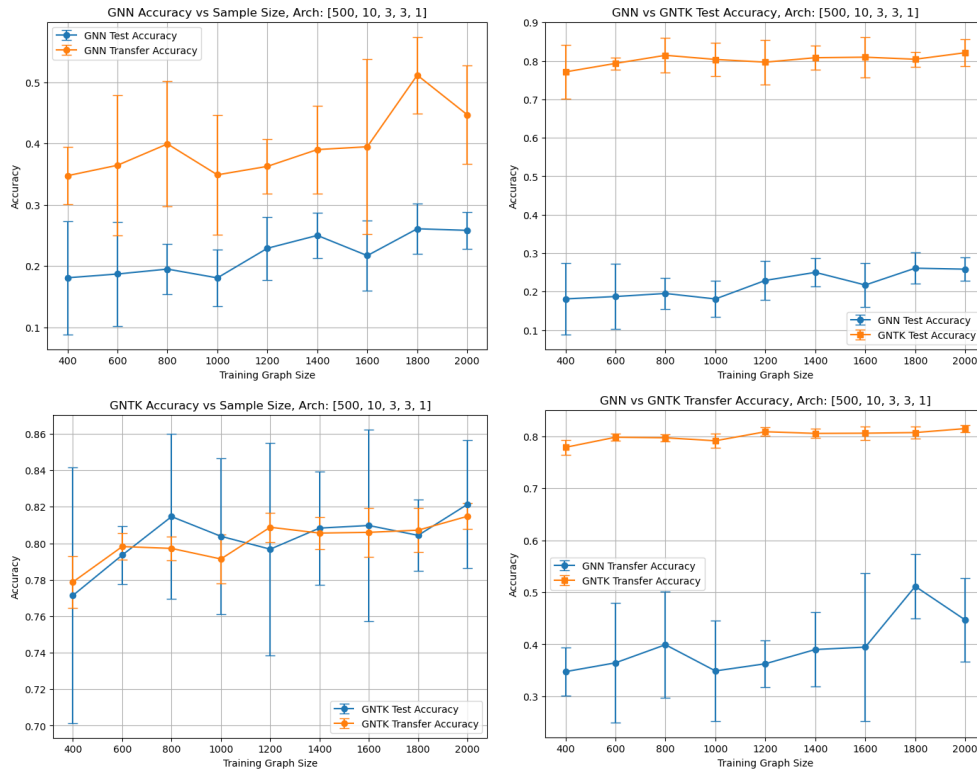


Figure 3: Results on PubMed dataset with num\_layers=3, and num\_hidden\_layers=10

All the experiments were done on an Intel i7-12700h CPU and NVIDIA GeForce RTX 3060 GPU.

## Conclusions

- The GNN models do not give sufficiently good accuracy in any of the datasets. Possibly for the following reason, a secondary goal of our project was to show that the GNN in the infinite width limit, and when trained with a low learning rate for a large number of epochs, should converge to the GNTK. Thus, we have deliberately chosen a low learning rate and a lot of different hidden layer sizes. But in doing so, we were limited in the max epochs to train each model, and not having a good GPU doesn't help.
- On the other hand, the GNTK shows very good accuracy for all the datasets. This accuracy also slightly increases with an increase in the subgraph sample size.
- We also observe high correlation in the GNTK test accuracy and the GNTK transfer accuracy for all the datasets.
- These align with the main idea of this project, that training a model on a small subgraph can be used to predict data on test sets of the entire graph.

## Contributions

- Shivam Kumar: Changing the GNN structure based on new GNTK, generating plots, and report writing
- Lovesh Mahar: Debugging initial GNTK code, initial rewrite of GNN.py, report writing
- Nihar Hingrajia: Integrating du et al's GNTK into this codebase, report writing
- Suryansh Srijan: Debugging initial GNTK code, initial rewrite of main.py and Kernel.py, running tests

## Cora

Table 1: GNN Test Accuracy (Mean  $\pm$  Std) Across Architectures and Sample Sizes

| Sample Size | Architecture    |                |                 |                 |                 |                 |                 |                |
|-------------|-----------------|----------------|-----------------|-----------------|-----------------|-----------------|-----------------|----------------|
|             | Arch 1          | Arch 2         | Arch 3          | Arch 4          | Arch 5          | Arch 6          | Arch 7          | Arch 8         |
| 400         | 20.4 $\pm$ 3.8  | 20.9 $\pm$ 5.2 | 21.6 $\pm$ 4.1  | 16.2 $\pm$ 3.2  | 20.7 $\pm$ 3.8  | 15.7 $\pm$ 1.6  | 18.0 $\pm$ 5.3  | 12.3 $\pm$ 2.2 |
| 600         | 23.7 $\pm$ 6.2  | 20.5 $\pm$ 4.7 | 28.9 $\pm$ 6.0  | 14.3 $\pm$ 3.7  | 27.7 $\pm$ 6.8  | 16.5 $\pm$ 5.6  | 14.8 $\pm$ 4.3  | 10.9 $\pm$ 2.1 |
| 800         | 22.9 $\pm$ 10.1 | 16.2 $\pm$ 4.3 | 23.5 $\pm$ 11.2 | 15.8 $\pm$ 3.8  | 20.8 $\pm$ 7.7  | 17.4 $\pm$ 3.4  | 18.0 $\pm$ 4.6  | 11.9 $\pm$ 1.2 |
| 1000        | 22.9 $\pm$ 7.9  | 17.8 $\pm$ 2.7 | 25.9 $\pm$ 11.8 | 21.8 $\pm$ 4.9  | 25.2 $\pm$ 8.0  | 19.2 $\pm$ 4.8  | 18.0 $\pm$ 1.0  | 13.6 $\pm$ 3.4 |
| 1200        | 19.3 $\pm$ 5.2  | 20.3 $\pm$ 8.7 | 18.4 $\pm$ 5.8  | 20.4 $\pm$ 7.8  | 19.5 $\pm$ 6.8  | 18.1 $\pm$ 5.3  | 18.7 $\pm$ 5.8  | 16.2 $\pm$ 3.9 |
| 1400        | 23.3 $\pm$ 8.5  | 20.7 $\pm$ 7.1 | 21.3 $\pm$ 10.2 | 21.8 $\pm$ 11.1 | 22.6 $\pm$ 12.5 | 21.0 $\pm$ 8.2  | 20.5 $\pm$ 8.2  | 15.2 $\pm$ 4.4 |
| 1600        | 20.4 $\pm$ 6.6  | 27.9 $\pm$ 9.2 | 18.4 $\pm$ 7.7  | 13.4 $\pm$ 2.3  | 18.0 $\pm$ 7.2  | 13.3 $\pm$ 2.2  | 17.8 $\pm$ 6.8  | 19.5 $\pm$ 4.4 |
| 1800        | 34.6 $\pm$ 7.6  | 31.9 $\pm$ 3.3 | 30.2 $\pm$ 7.0  | 24.4 $\pm$ 13.3 | 26.7 $\pm$ 5.5  | 27.0 $\pm$ 10.1 | 36.7 $\pm$ 11.5 | 17.7 $\pm$ 3.4 |
| 2000        | 33.2 $\pm$ 7.7  | 37.9 $\pm$ 4.2 | 25.4 $\pm$ 5.2  | 28.3 $\pm$ 11.6 | 23.5 $\pm$ 7.3  | 27.1 $\pm$ 11.3 | 26.4 $\pm$ 10.4 | 20.6 $\pm$ 4.3 |

Table 2: GNN Transfer Accuracy (Mean  $\pm$  Std) Across Architectures and Sample Sizes

| Sample Size | Architecture    |                 |                 |                 |                 |                 |                 |                |
|-------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|----------------|
|             | Arch 1          | Arch 2          | Arch 3          | Arch 4          | Arch 5          | Arch 6          | Arch 7          | Arch 8         |
| 400         | $27.6 \pm 8.3$  | $27.9 \pm 10.9$ | $29.5 \pm 7.9$  | $17.0 \pm 4.9$  | $26.5 \pm 9.7$  | $16.8 \pm 5.8$  | $19.6 \pm 6.3$  | $14.7 \pm 6.3$ |
| 600         | $26.4 \pm 10.6$ | $18.5 \pm 8.0$  | $34.6 \pm 13.0$ | $15.5 \pm 9.4$  | $32.9 \pm 13.4$ | $23.0 \pm 8.5$  | $19.6 \pm 2.4$  | $11.3 \pm 2.3$ |
| 800         | $26.7 \pm 16.8$ | $14.3 \pm 3.0$  | $25.8 \pm 14.5$ | $17.3 \pm 8.5$  | $22.2 \pm 9.5$  | $21.1 \pm 8.5$  | $20.1 \pm 7.6$  | $14.0 \pm 3.0$ |
| 1000        | $24.8 \pm 13.3$ | $15.2 \pm 3.1$  | $29.2 \pm 19.7$ | $20.3 \pm 7.2$  | $25.9 \pm 13.1$ | $19.0 \pm 5.7$  | $22.2 \pm 6.7$  | $12.5 \pm 4.0$ |
| 1200        | $21.9 \pm 8.4$  | $23.8 \pm 12.0$ | $21.4 \pm 8.6$  | $22.9 \pm 10.5$ | $22.0 \pm 9.3$  | $21.8 \pm 9.1$  | $19.4 \pm 5.9$  | $17.5 \pm 4.2$ |
| 1400        | $24.4 \pm 9.1$  | $20.2 \pm 7.4$  | $23.9 \pm 12.4$ | $25.2 \pm 14.5$ | $25.3 \pm 14.7$ | $23.8 \pm 11.0$ | $21.5 \pm 8.5$  | $16.6 \pm 5.7$ |
| 1600        | $20.6 \pm 9.0$  | $27.9 \pm 11.2$ | $19.4 \pm 9.8$  | $13.4 \pm 2.2$  | $18.8 \pm 8.7$  | $13.4 \pm 2.1$  | $18.9 \pm 8.7$  | $20.2 \pm 6.9$ |
| 1800        | $36.0 \pm 9.7$  | $33.2 \pm 4.1$  | $31.3 \pm 8.5$  | $25.4 \pm 15.6$ | $28.5 \pm 7.1$  | $27.8 \pm 11.6$ | $39.0 \pm 14.6$ | $18.0 \pm 4.5$ |
| 2000        | $34.2 \pm 8.5$  | $38.9 \pm 6.9$  | $26.6 \pm 6.5$  | $29.7 \pm 12.8$ | $25.1 \pm 8.3$  | $28.7 \pm 12.3$ | $27.4 \pm 11.1$ | $22.0 \pm 4.2$ |

Table 3: GNTK Test Accuracy (Mean  $\pm$  Std) Across Architectures and Sample Sizes[illegible]Table 4: GNTK Transfer Accuracy (Mean  $\pm$  Std) Across Architectures and Sample Sizes[illegible]

Table 5: GNN Test Accuracy (Mean  $\pm$  Std) Across Architectures and Sample Sizes

| Sample Size | Architecture   |                |                |                |                |                |                |                |
|-------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
|             | Arch 1         | Arch 2         | Arch 3         | Arch 4         | Arch 5         | Arch 6         | Arch 7         | Arch 8         |
| 400         | $15.4 \pm 3.5$ | $14.4 \pm 2.1$ | $14.0 \pm 2.5$ | $9.4 \pm 2.8$  | $14.0 \pm 5.8$ | $10.1 \pm 2.8$ | $9.8 \pm 2.3$  | $8.6 \pm 2.0$  |
| 600         | $19.6 \pm 5.6$ | $18.0 \pm 2.5$ | $22.4 \pm 5.6$ | $15.5 \pm 4.6$ | $22.2 \pm 5.3$ | $12.8 \pm 2.9$ | $13.8 \pm 3.0$ | $11.3 \pm 2.6$ |
| 800         | $20.5 \pm 6.4$ | $19.6 \pm 6.0$ | $21.7 \pm 6.6$ | $19.0 \pm 3.2$ | $22.7 \pm 5.3$ | $20.4 \pm 4.8$ | $17.8 \pm 3.4$ | $13.8 \pm 1.5$ |
| 1000        | $18.5 \pm 3.6$ | $16.0 \pm 4.7$ | $20.6 \pm 6.2$ | $16.9 \pm 5.0$ | $20.3 \pm 4.9$ | $16.7 \pm 2.1$ | $17.6 \pm 5.1$ | $12.2 \pm 2.1$ |
| 1200        | $20.3 \pm 2.9$ | $18.8 \pm 3.2$ | $22.7 \pm 4.9$ | $21.0 \pm 3.4$ | $22.3 \pm 4.1$ | $18.5 \pm 2.8$ | $21.0 \pm 2.4$ | $13.9 \pm 3.3$ |
| 1400        | $17.4 \pm 2.1$ | $17.3 \pm 2.2$ | $17.4 \pm 2.1$ | $17.9 \pm 1.6$ | $17.4 \pm 2.1$ | $18.6 \pm 1.5$ | $20.4 \pm 1.8$ | $14.0 \pm 2.8$ |
| 1600        | $21.9 \pm 5.3$ | $16.8 \pm 2.0$ | $20.6 \pm 5.6$ | $21.7 \pm 5.3$ | $22.2 \pm 5.3$ | $22.4 \pm 5.4$ | $26.1 \pm 4.8$ | $13.9 \pm 4.0$ |
| 1800        | $20.0 \pm 2.9$ | $19.1 \pm 1.8$ | $18.2 \pm 1.2$ | $20.7 \pm 4.0$ | $20.3 \pm 4.0$ | $20.3 \pm 2.7$ | $21.5 \pm 3.4$ | $16.8 \pm 2.8$ |
| 2000        | $19.7 \pm 2.8$ | $17.4 \pm 1.4$ | $17.4 \pm 1.3$ | $17.6 \pm 1.3$ | $17.4 \pm 1.3$ | $17.9 \pm 1.6$ | $21.6 \pm 4.3$ | $17.5 \pm 2.2$ |

Table 6: GNN Transfer Accuracy (Mean  $\pm$  Std) Across Architectures and Sample Sizes

| Sample Size | Architecture   |                |                 |                |                 |                |                |                |
|-------------|----------------|----------------|-----------------|----------------|-----------------|----------------|----------------|----------------|
|             | Arch 1         | Arch 2         | Arch 3          | Arch 4         | Arch 5          | Arch 6         | Arch 7         | Arch 8         |
| 400         | $30.9 \pm 6.8$ | $28.2 \pm 6.2$ | $32.1 \pm 10.9$ | $18.0 \pm 1.6$ | $27.8 \pm 12.0$ | $14.8 \pm 4.9$ | $20.6 \pm 2.1$ | $14.8 \pm 2.7$ |
| 600         | $33.7 \pm 8.4$ | $35.7 \pm 9.4$ | $39.9 \pm 7.7$  | $27.2 \pm 7.1$ | $37.5 \pm 9.1$  | $23.0 \pm 3.5$ | $26.4 \pm 5.4$ | $19.6 \pm 1.6$ |
| 800         | $25.0 \pm 5.7$ | $24.4 \pm 5.4$ | $28.5 \pm 8.6$  | $24.2 \pm 5.4$ | $29.0 \pm 9.6$  | $27.1 \pm 6.6$ | $24.6 \pm 3.2$ | $15.2 \pm 4.4$ |
| 1000        | $24.7 \pm 3.9$ | $22.1 \pm 7.1$ | $28.4 \pm 8.3$  | $23.2 \pm 7.7$ | $27.6 \pm 5.7$  | $22.1 \pm 3.4$ | $26.4 \pm 8.3$ | $18.2 \pm 5.7$ |
| 1200        | $25.1 \pm 7.3$ | $21.5 \pm 4.7$ | $29.0 \pm 9.2$  | $27.6 \pm 7.8$ | $27.8 \pm 7.9$  | $24.9 \pm 6.3$ | $27.4 \pm 5.1$ | $16.9 \pm 2.5$ |
| 1400        | $18.3 \pm 0.0$ | $18.3 \pm 0.0$ | $18.3 \pm 0.0$  | $18.7 \pm 0.8$ | $18.3 \pm 0.0$  | $19.2 \pm 1.0$ | $22.2 \pm 3.1$ | $18.8 \pm 4.2$ |
| 1600        | $25.3 \pm 6.9$ | $18.8 \pm 1.1$ | $24.1 \pm 7.1$  | $25.2 \pm 7.1$ | $25.8 \pm 6.8$  | $25.7 \pm 7.0$ | $30.0 \pm 6.7$ | $19.0 \pm 5.9$ |
| 1800        | $20.5 \pm 4.1$ | $19.3 \pm 1.5$ | $18.7 \pm 0.8$  | $21.4 \pm 6.2$ | $21.2 \pm 5.9$  | $20.6 \pm 4.1$ | $22.7 \pm 5.5$ | $17.3 \pm 3.3$ |
| 2000        | $19.6 \pm 1.4$ | $18.3 \pm 0.0$ | $18.3 \pm 0.0$  | $18.4 \pm 0.1$ | $18.3 \pm 0.0$  | $18.8 \pm 0.9$ | $22.3 \pm 4.0$ | $19.9 \pm 3.3$ |

Table 7: GNTK Test Accuracy (Mean  $\pm$  Std) Across Architectures and Sample Sizes[illegible]Table 8: GNTK Transfer Accuracy (Mean  $\pm$  Std) Across Architectures and Sample Sizes[illegible]

